```
import logging

logger = logging.getLogger()#creates a root logger

logger.info("This is an info message")
logger.warning("This is a warning message")#Root Handler prints from Warning level messages
```

```
WARNING:root:This is a warning message
```

```
import logging

logging.lastResort = None
logger1 = logging.getLogger("example")
logger1.propagate = False #will prevent propagating upwards

logger1.info("This is an info message")
logger1.warning("This is a warning message")
```

```
No handlers could be found for logger "example"
```

```
import logging

logger1 = logging.getLogger("example20")
logger1.propagate = False #will prevent propagating upwards

logger1.info("This is an info message")
logger1.warning("This is a warning message")
```

```
import logging

logging.lastResort = None
logger1 = logging.getLogger("example8")

logger1.info("This is an info message")
logger1.warning("This is a warning message")
```

```
WARNING:example8:This is a warning message
```

```
import logging

logger1 = logging.getLogger("example17")

logger1.debug("This is a debug message")
logger1.info("This is a info message")
logger1.warning("This is a warning message")
logger1.error("This is an error message")
logger1.critical("This is a critical message")

try:
  raise ValueError("This is Value Eroor")
except ValueError:
  logger1.exception("This is an exception message")
```

```
WARNING:example17:This is a warning message
ERROR:example17:This is an error message
CRITICAL:example17:This is a critical message
ERROR:example17:This is an exception message
Traceback (most recent call last):
  File "/tmp/ipython-input-1714017207.py", line 12, in <cell line: 0>
    raise ValueError("This is Value Eroor")
ValueError: This is Value Eroor
```

```
import logging

logger1 = logging.getLogger("example7")

logger1.info("This is an info message")
logger1.warning("This is a warning message")
```

```
WARNING:example7:This is a warning message
```

```
import logging

mylogger = logging.getLogger("example12")

mylogger.setLevel(logging.INFO)#set the mylogger's overall level

stream_handler = logging.StreamHandler()
mylogger.addHandler(stream_handler)

stream_handler.setLevel(logging.ERROR)# set the level for the stream_handler

mylogger.propagate = False#will prevent propagating upwards

mylogger.debug("This is a debug message")
mylogger.info("This is an info message")
mylogger.warning("This is a warning message")
mylogger.error("This is an error message")
mylogger.critical("This is a critical message")
```

```
This is an error message
This is a critical message
```

```
import logging

logger5 = logging.getLogger("example2")

logger5.setLevel(logging.INFO)#set the logger's overall level

class CustomFilter(logging.Filter):
  def filter(self,record:logging.LogRecord):
    return record. levelno >= logging.WARNING

logger5.addFilter(CustomFilter())

logger5.debug("This is a debug message")
logger5.info("This is an info message")
logger5.warning("This is a warning message")
logger5.error("This is an error message")
logger5.critical("This is a critical message")
```

```
WARNING:example2:This is a warning message
ERROR:example2:This is an error message
CRITICAL:example2:This is a critical message
```

```
import logging

logger5 = logging.getLogger("example15")

logger5.setLevel(logging.INFO)#set the logger's overall level

stream_handler = logging.StreamHandler()
logger5.addHandler(stream_handler)

stream_handler.setLevel(logging.WARNING)

stream_formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
stream_handler.setFormatter(stream_formatter)


class CustomFilter(logging.Filter):
  def filter(self,record:logging.LogRecord):
    return record.levelno >= logging.WARNING

logger5.addFilter(CustomFilter())

logger5.debug("This is a debug message")
logger5.info("This is an info message")
logger5.warning("This is a warning message")
logger5.error("This is an error message")
logger5.critical("This is a critical message")
```

```
2025-11-27 05:55:11,670 - example15 - WARNING - This is a warning message
2025-11-27 05:55:11,671 - example15 - ERROR - This is an error message
2025-11-27 05:55:11,671 - example15 - ERROR - This is an error message
2025-11-27 05:55:11,676 - example15 - CRITICAL - This is a critical message
2025-11-27 05:55:11,676 - example15 - CRITICAL - This is a critical message
```

```
import logging

logger5 = logging.getLogger("example16")

logger5.setLevel(logging.INFO)#set the logger's overall level

stream_handler = logging.StreamHandler()
logger5.addHandler(stream_handler)

stream_handler.setLevel(logging.WARNING)

stream_formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
stream_handler.setFormatter(stream_formatter)


class CustomFilter(logging.Filter):
  def filter(self,record:logging.LogRecord):
    return record.levelno >= logging.WARNING

logger5.addFilter(CustomFilter())
logger5.propagate = False #will prevent propagating upwards

logger5.debug("This is a debug message")
logger5.info("This is an info message")
logger5.warning("This is a warning message")
logger5.error("This is an error message")
logger5.critical("This is a critical message")
```

```
2025-11-27 06:53:30,662 - example16 - WARNING - This is a warning message
2025-11-27 06:53:30,664 - example16 - ERROR - This is an error message
2025-11-27 06:53:30,665 - example16 - CRITICAL - This is a critical message
```

```
import logging

logger5 = logging.getLogger("example21")

logger5.setLevel(logging.INFO)#set the logger's overall level

stream_handler = logging.StreamHandler()
logger5.addHandler(stream_handler)

stream_handler.setLevel(logging.WARNING)

stream_formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
stream_handler.setFormatter(stream_formatter)

def custom_Filter(record:logging.LogRecord):
  return "oops" in record.getMessage()

logger5.addFilter(custom_Filter)


class CustomFilter(logging.Filter):
  def filter(self,record:logging.LogRecord):
    return record.levelno >= logging.WARNING

logger5.addFilter(CustomFilter())
logger5.propagate = False #will prevent propagating upwards

logger5.debug("This is a debug message")
logger5.info("This is an info message")
logger5.warning("This is a warning message")
logger5.warning("This is an oops message")
logger5.error("This is an error message")
logger5.critical("This is a critical message")
```

```
2025-11-27 07:33:24,597 - example21 - WARNING - This is an oops message
```

Everything starts with a logger object. Every logger object can be assigned a log level. There are totally 5 log levels(DEBUG, INFO, WARNING,ERROR and CRITICAL).

The main purpose of a handler is to direct the log record to the specified output(streamhandler or filehandler).

If no handler is configured for a logger object or its ancestors, the log record is sent to the last resort handler. Its log level is WARNING and it has no formatting.

Loglevels can be set for the entire logger and also can be set differently for different handlers.

If the log level is not set for a logger object, it gets the log level from its first ancestor for whom the log level is explicitly set.

Handler's output can be customized by setting a formatter.

Filters can be associated with logger object or to handlers to further control which log records get displayed/stored.

```
import logging

logger = logging.getLogger("example3")

logger.setLevel(logging.INFO)

logger.info("This is an info message")
logger.warning("This is a warning message")
```

```
INFO:example3:This is an info message
WARNING:example3:This is a warning message
```

```
import logging

# Get a logger instance
logger8 = logging.getLogger("example4")

# Log various messages
logger8.debug("This is a debug message.")
logger8.info("This is an info message.")
logger8.warning("This is a warning message.")
logger8.error("This is an error message.")
logger8.critical("This is a critical message.")
```

```
WARNING:example4:This is a warning message.
ERROR:example4:This is an error message.
CRITICAL:example4:This is a critical message.
```

```
import logging

root_logger = logging.getLogger()
print(f"Root logger level: {root_logger.level}")

child_logger = logging.getLogger("example5")

stream_handler = logging.StreamHandler()
child_logger.addHandler(stream_handler)

print(f"Child logger level: {child_logger.level}")

child_logger.info("This will not be logged")
child_logger.warning("This is a warning message")
```

```
This is a warning message
WARNING:example5:This is a warning message
Root logger level: 30
Child logger level: 0
```

```
import logging

root_logger = logging.getLogger()
print(f"Root logger level: {root_logger.level}")

child_logger = logging.getLogger("example10")

stream_handler = logging.StreamHandler()
child_logger.addHandler(stream_handler)

print(f"Child logger level: {child_logger.level}")

child_logger.propagate = False

child_logger.info("This will not be logged")
child_logger.warning("This is a warning message")
```

```
This is a warning message
Root logger level: 30
Child logger level: 0
```

```
import logging

# Get a logger instance
logger8 = logging.getLogger("example13")
logger8.setLevel(logging.INFO)

stream_handler = logging.StreamHandler()
logger8.addHandler(stream_handler)

file_handler = logging.FileHandler("app.log")
logger8.addHandler(file_handler)

stream_handler.setLevel(logging.ERROR)
file_handler.setLevel(logging.CRITICAL)

stream_formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
stream_handler.setFormatter(stream_formatter)

# Log various messages
logger8.debug("This is a debug message.")
logger8.info("This is an info message.")
logger8.warning("This is a warning message.")
logger8.error("This is an error message.")
logger8.critical("This is a critical message.")
```

```
INFO:example13:This is an info message.
WARNING:example13:This is a warning message.
2025-11-27 05:24:03,147 - example13 - ERROR - This is an error message.
ERROR:example13:This is an error message.
2025-11-27 05:24:03,149 - example13 - CRITICAL - This is a critical message.
CRITICAL:example13:This is a critical message.
```

```
import logging

# Get a logger instance
logger8 = logging.getLogger("example14")

stream_handler = logging.StreamHandler()
logger8.addHandler(stream_handler)

file_handler = logging.FileHandler("app.log")
logger8.addHandler(file_handler)

stream_handler.setLevel(logging.ERROR)
file_handler.setLevel(logging.CRITICAL)

stream_formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
stream_handler.setFormatter(stream_formatter)

# Log various messages
logger8.debug("This is a debug message.")
logger8.info("This is an info message.")
logger8.warning("This is a warning message.")
logger8.error("This is an error message.")
```

```
WARNING:example14:This is a warning message.
2025-11-27 05:27:03,617 - example14 - ERROR - This is an error message.
ERROR:example14:This is an error message.
```

```
import logging

# Get a logger instance
logger8 = logging.getLogger("example15")

stream_handler = logging.StreamHandler()
logger8.addHandler(stream_handler)

file_handler = logging.FileHandler("app.log")
logger8.addHandler(file_handler)

stream_handler.setLevel(logging.ERROR)
file_handler.setLevel(logging.CRITICAL)

stream_formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s - %(message)s")
stream_handler.setFormatter(stream_formatter)

logger8.propagate = False
```

```
# Log various messages
logger8.debug("This is a debug message.")
logger8.info("This is an info message.")
logger8.warning("This is a warning message.")
logger8.error("This is an error message.")
logger8.critical("This is a critical message")
```

```
2025-11-27 05:31:35,529 - example15 - ERROR - This is an error message.
2025-11-27 05:31:35,530 - example15 - CRITICAL - This is a critical message
```

```
Start coding or generate with AI.
```

```
import logging.config
import logging

config_dict = {
    "version": 1,
    "formatters": {
        "default": {
            "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s",
        },
    },
    "handlers": {
        "console": {
            "class": "logging.StreamHandler",
            "formatter": "default"
          },

        "file": {
            "class": "logging.FileHandler",
            "filename": "app.log",
            "formatter": "default",
         },
     },

    "loggers": {
        "my_app": {
           "level": "WARNING",
           "handlers": ["file"],
           "propagate": False,
     },
       "myapp_database": {
           "level": "WARNING",
           "handlers": ["console"],
    },
  },
}

logging.config.dictConfig(config_dict)

logger = logging.getLogger("myapp_database")
logger.warning("This is a warning message")
```

```
2025-11-27 09:17:45,147 - myapp_database - WARNING - This is a warning message
WARNING:myapp_database:This is a warning message
```

```
import logging

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(name)s - %(message)s',
                    force=True)

logger = logging.getLogger()

logger.debug("This is a debug message")
logger.info("This is an informational message.")
logger.warning("This is a warning message.")
```

```
2025-11-27 10:04:02,904 - INFO - root - This is an informational message.
2025-11-27 10:04:02,907 - WARNING - root - This is a warning message.
```